

End-to-End Machine Learning Pipeline

Telco Customer Churn Prediction

Task 16 — PKCERT AI & Software Development Internship

Author: Abdul Rehman

1. Project Overview

This project builds a complete, end-to-end machine learning pipeline to predict customer churn for a telecommunications company — i.e., whether a customer will cancel their subscription. Churn prediction is a classic real-world business ML problem: catching at-risk customers early lets a company target retention offers proactively instead of losing revenue reactively.

The project covers the full ML lifecycle: dataset selection, cleaning, exploratory data analysis (EDA), feature engineering, pipeline-based preprocessing, multi-model training and comparison, hyperparameter tuning, rigorous evaluation, and model persistence — all inside a single well-documented Jupyter notebook.

2. Dataset (Part A — Dataset Selection & Data Preparation, 25 Marks)

2.1 Dataset Description

Source: Telco Customer Churn dataset (originally an IBM Sample Data Set), available on Kaggle, included locally at `data/Telco-Customer-Churn.csv`.

- Rows: 7,043 customers
- Columns: 21 (20 features + target)
- Target: Churn (Yes/No) — imbalanced, approximately 26.5% churn rate
- Features: Demographics (gender, senior citizen, partner, dependents), account information (tenure, contract type, payment method, billing), and subscribed services (phone, internet, streaming, security add-ons, etc.)

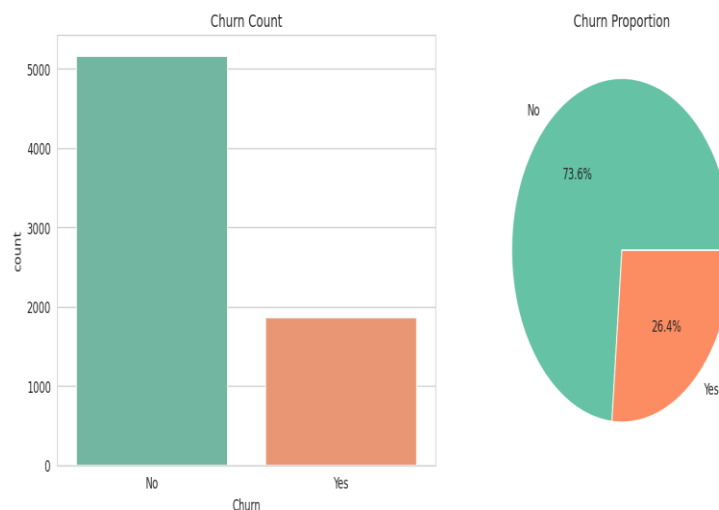


Figure 1. Class balance of the target variable (Churn).

2.2 Data Cleaning

- Discovered that `TotalCharges` was stored as text due to 11 blank-string values, all belonging to brand-new customers (`tenure == 0`). Converted to numeric and imputed as 0.
- Dropped the non-predictive `customerID` identifier column.
- Verified no duplicate records.

2.3 Exploratory Data Analysis

Visualized target class balance, numeric feature distributions by churn status, correlations, and churn rate broken down by key categorical drivers (contract type, internet service, payment method, senior citizen status).

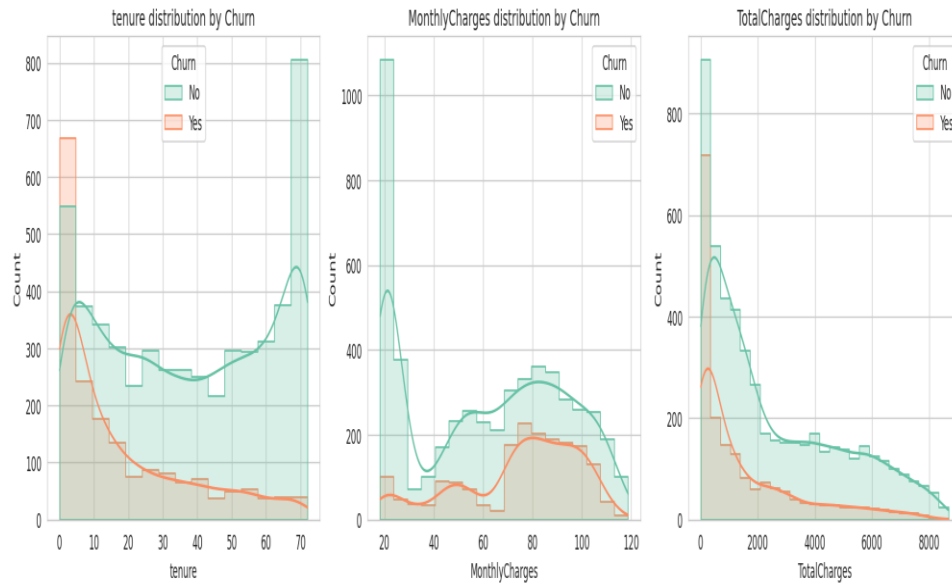


Figure 2. Numeric feature distributions split by churn status.

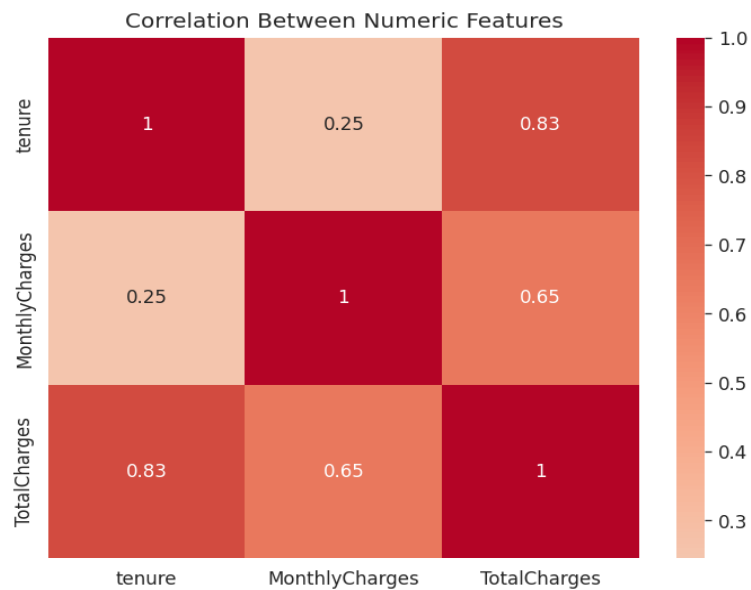


Figure 3. Correlation heatmap of numeric features.

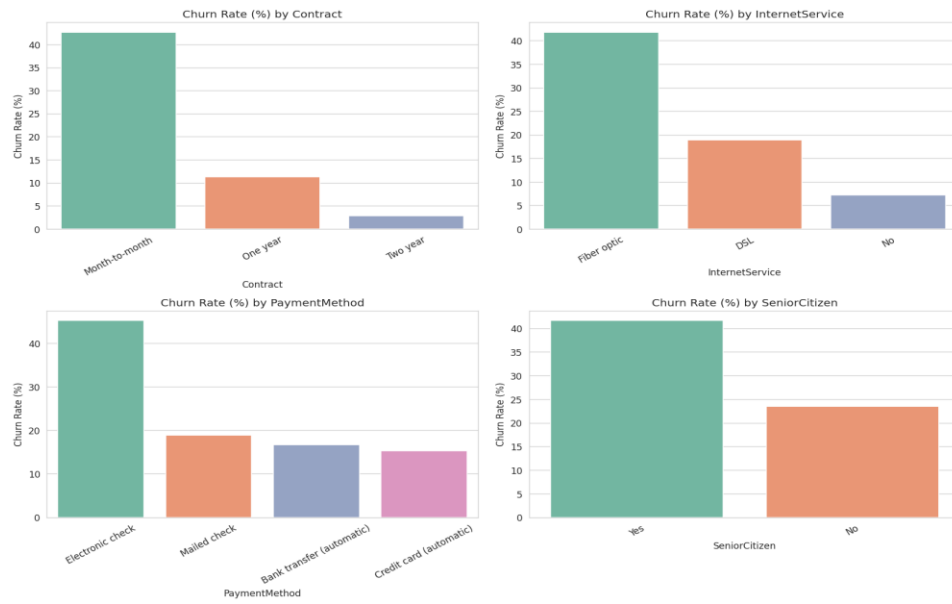


Figure 4. Churn rate by key categorical drivers.

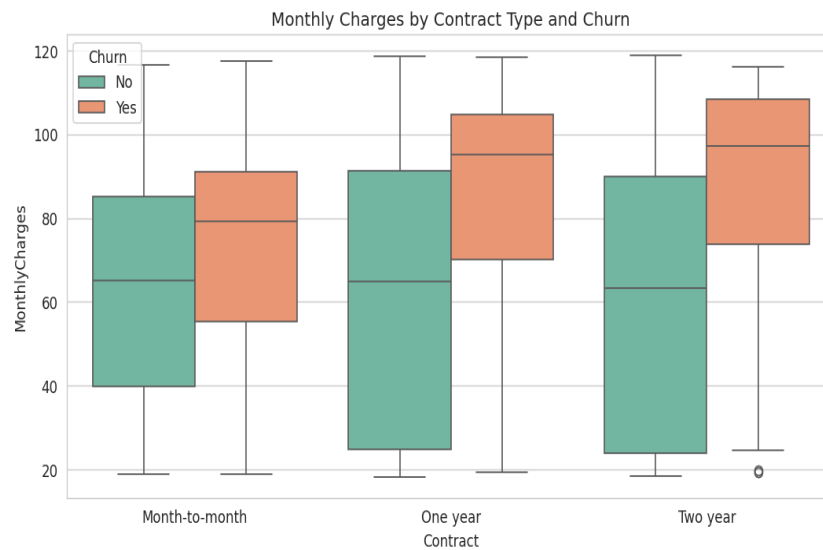


Figure 5. Monthly charges distribution by contract type.

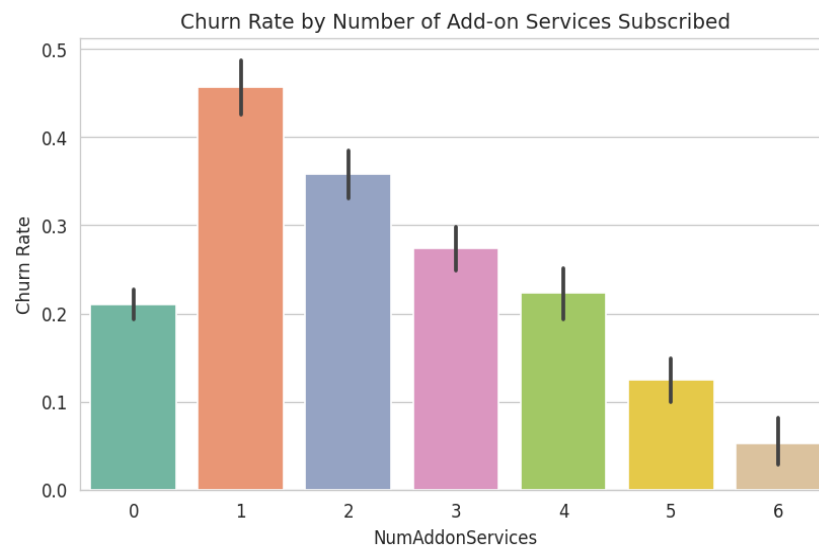


Figure 6. Add-on service count vs. churn.

Key finding: month-to-month contracts, low tenure, high monthly charges, and electronic-check payment are all associated with substantially higher churn.

2.4 Feature Engineering

- `TenureGroup`: bucketed tenure into lifecycle stages (0–1yr, 1–2yr, 2–4yr, 4yr+).
- `NumAddonServices`: count of subscribed add-on services (security, backup, tech support, streaming), used as a proxy for customer engagement.
- `AvgMonthlySpend`: $\text{TotalCharges} / \text{tenure}$, a smoothed spend-rate feature.

3. Model Development (Part B — Model Development, 35 Marks)

3.1 Preprocessing & Pipeline Design

Built a `ColumnTransformer` (`StandardScaler` for numeric features, `OneHotEncoder` for categorical features) wrapped inside a scikit-learn `Pipeline` together with the classifier — ensuring no data leakage between train and test sets.

3.2 Model Training & Comparison

Trained and compared three algorithms with 5-fold stratified cross-validation (ROC-AUC scoring):

- Logistic Regression
- Random Forest
- XGBoost

Each model was configured to handle class imbalance (`class_weight='balanced'` / `scale_pos_weight`).

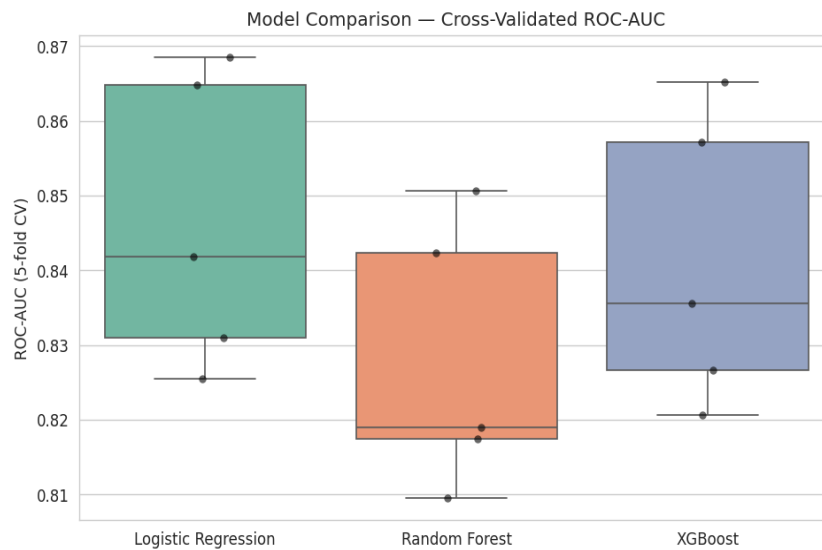


Figure 7. Cross-validated ROC-AUC comparison across candidate models.

3.3 Hyperparameter Tuning

Selected the best-performing model family and further tuned it with GridSearchCV.

4. Model Evaluation (Part C — Model Evaluation, 20 Marks)

4.1 Evaluation Methodology

Evaluated the final tuned pipeline on a held-out 20% test set using Accuracy, Precision, Recall, F1-score, and ROC-AUC, plus a confusion matrix, ROC curve, and feature importance plot.

4.2 Results

Metric	Test Set Score
Accuracy	0.738
Precision (Churn)	0.504
Recall (Churn)	0.774
F1-score (Churn)	0.610
ROC-AUC	0.840

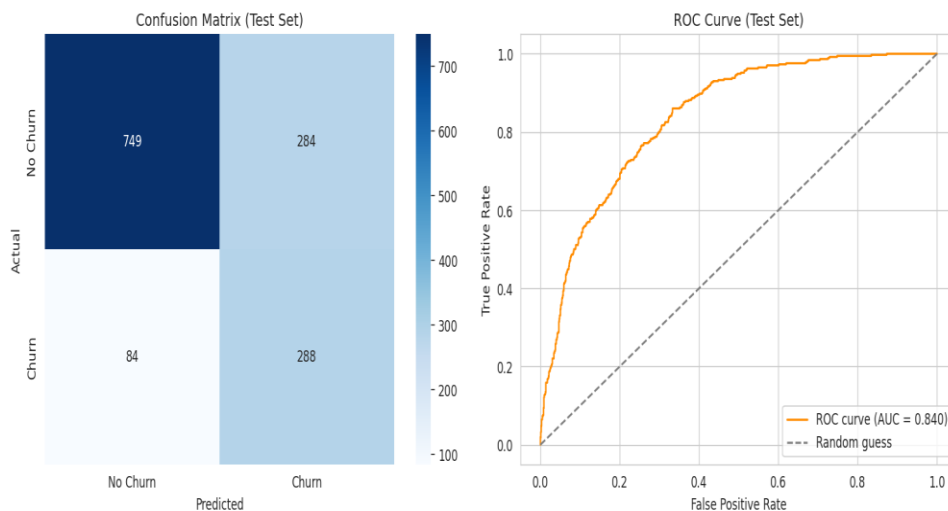


Figure 8. Confusion matrix and ROC curve on the held-out test set.

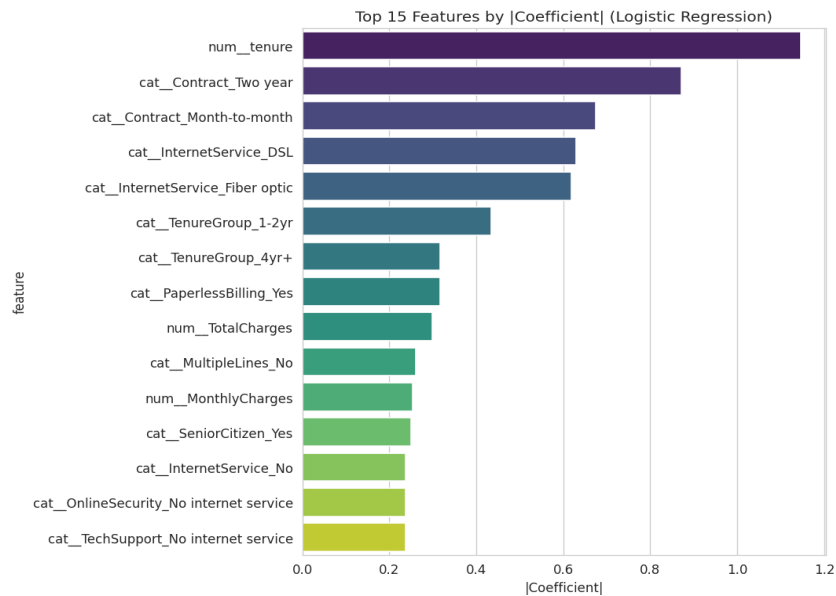


Figure 9. Feature importances of the final tuned model.

4.3 Interpretation

Logistic Regression (tuned via GridSearchCV) outperformed Random Forest and XGBoost on cross-validated ROC-AUC — a reasonable outcome given that churn drivers in this dataset (contract type, tenure, charges) behave in a fairly linear/additive way, and the simpler model generalizes slightly better on this data than the more complex ensembles.

Class-imbalance handling was tuned to favor recall on the churn class, since in a retention context, missing an actual churner (false negative) is more costly to the business than a false positive (an unnecessary but harmless retention offer).

4.4 Strengths & Limitations

Strengths

- Leakage-safe end-to-end pipeline
- Imbalance-aware training
- Multiple models fairly compared via cross-validation
- Interpretable feature importances aligned with business intuition
- Reproducible model persistence

Limitations

- Static dataset snapshot (no behavioral time-series)
- Default 0.5 classification threshold not cost-tuned to business economics
- Single train/test split used for final reporting
- No access to potentially stronger real-world signals (support call logs, complaint history, competitor offers)

5. Project Documentation (Part D — Project Documentation, 20 Marks)

5.1 Repository Structure

task16/

- Task16_Churn_Pipeline.ipynb – Main notebook (full pipeline, executed with outputs)
- README.md – Project documentation
- requirements.txt – Python dependencies

- data/Telco-Customer-Churn.csv – Raw dataset
- figures/ – All EDA and evaluation plots (PNG)
- models/churn_pipeline.joblib – Final trained pipeline (preprocessing + model)

5.2 How to Run

```
pip install -r requirements.txt
```

```
jupyter notebook Task16_Churn_Pipeline.ipynb
```

To reuse the saved model on new data:

```
import joblib
```

```
model = joblib.load('models/churn_pipeline.joblib')
```

```
predictions = model.predict(new_customer_dataframe)
```

```
probabilities = model.predict_proba(new_customer_dataframe)[: , 1]
```

new_customer_dataframe must contain the same raw columns used during training, including the engineered features TenureGroup, NumAddonServices, and AvgMonthlySpend.

6. Conclusion

The final pipeline offers a practical, interpretable tool for identifying at-risk customers, allowing the business to prioritize retention efforts on month-to-month, low-tenure, high-monthly-charge customers — consistently flagged as highest-risk by both the EDA and the model's feature importances.